

A compact precursor table for very large spectral libraries

Pioneer.jl, branch `feat/immunopeptidomics`

9 October 2026

Summary

The first full build of the human non-enzymatic 8–12mer library (299.6M precursors, charges 1–3, ≤ 1 Met-ox) finished under a 32 GB memory cap, and its `precursors_table.arrow` is 34 GB (113 bytes per precursor). Almost half of that is five string columns, much of it redundant, and two columns are never read. This plan proposes a second precursor-table schema (“v2”): sequences as packed 5-bit residue codes, modifications as compact (position, site, mod-name ID) entries, protein information moved to per-peptide side tables, and the unread columns dropped. The estimated precursor table plus side tables is about 16 GB (**about half**).

The disk saving is not the main reason to do this. When SearchDIA loads a library, `SetPrecursors` builds a `String` key for every precursor’s accession set (to assign cross-validation folds by protein group), and the first output step ranks every sequence, mods string and accession string (`getPrecursorTextIds`). At 300M rows both materialize hundreds of millions of strings. With the IDs computed once at build time, both become a lookup. Older (v1) libraries keep loading unchanged, and SearchDIA’s outputs keep the same text.

1 Where the 34 GB goes

Measured per column on the final table of the 1% human non-enzymatic mock library (2.96M rows, 121 bytes per row), scaled to 299.6M rows. The real 300M file is 113 bytes per row (34 GB), so scaled figures are within about 7%.

Column	Bytes/row	At 300M	Content
<code>koina_sequence</code>	18.1	5.4 GB	sequence with [UNIMOD:x] tags; input to Koina
<code>sequence</code>	14.0	4.2 GB	peptide string (incl. 4-byte offset)
<code>accession_numbers</code>	10.1	3.0 GB	;-joined accessions, repeated on every row of a peptide
<code>structural_mods</code>	9.8	2.9 GB	e.g. (5,M,Unimod:35); 2,196 distinct values at 1%
<code>proteome_identifiers</code>	9.0	2.7 GB	;-joined proteome names; 11 distinct at 1%
<code>start_idx</code>	8.0	2.4 GB	list of protein start positions
<code>isotopic_mods, isotope_mods</code>	8.2	2.4 GB	always missing without isotope groups
<code>partner_precursor_idx</code>	8.1	2.4 GB	decoy/target partner row, stored as <code>Int64</code>
<code>collision_energy</code>	4.0	1.2 GB	the build’s one NCE on every row
6 numeric 4-byte columns	24.0	7.2 GB	<code>mz, irt, pair_id, entrapment_pair_id, base*_id</code>
8 one-byte columns	8.0	2.4 GB	charge, decoy, length, ...

Distinct values at 1%: 1.03M sequences for 2.96M rows (2.9 rows per sequence: charges and decoys); 615,808 `base_pep_id` (one per modified target form, shared by its decoy and entrapments), and `base_pep_id` determines `accession_numbers` and `start_idx` exactly (615,808 distinct pairs each).

2 Who reads each column

From `src/` (BuildSpecLib reads only its intermediate `precursors.arrow`, not the final table):

- `koina_sequence`: no reader of the final table. Needed only during the build (Koina requests).
- `collision_energy`: **no reader.** `getCollisionEnergy` is defined in `structs/SpectralLibrary/precursors.jl` and never called. SearchDIA’s intensities come from the calibrated NCE model, not this column.

- **isotopic_mods, isotope_mods:** output text (`precursorText.jl`, `writeCSVTables.jl`), protein quantification, text IDs.
- **sequence:** iRT refinement and MainSearch features (token counts), parameter tuning, chromatogram integration, protein inference and quantification, output text, text IDs.
- **structural_mods:** MainSearch features, iRT refinement, protein inference and quantification, output text, variable-mod counting for old libraries, text IDs.
- **accession_numbers: library load** (CV folds, `accession_numbers_to_pid`), protein inference, MaxLFQ, output text, text IDs.
- **proteome_identifiers:** MaxLFQ, species IDs. **start_idx:** output text only.

Collision energy. Today BuildSpecLib writes one value (`nce_params.nce`) on every row, and nothing reads it. A library could in principle carry different energies per precursor (an empirical library, or a build over several NCEs), so the column is not dropped for being constant: it is dropped because it has no reader. The build NCE stays in `config.json`. If a feature ever needs per-precursor energies, it comes back as a dictionary-encoded column (a handful of distinct values costs 1 byte per row).

partner_precursor_idx. A row position, which fits in UInt32 (4 bytes instead of 8; readers convert on read). Optional, -1.2 GB.

entrapment_pair_id. Left as is (1.2 GB, and it only duplicates `pair_id` in libraries without entrapments).

3 Proposed v2 schema

Sequences and mods are stored in compact binary forms that decode to today’s strings on demand. Everything about a precursor’s proteins is determined by its `base_pep_id` (measured: 615,808 distinct `base_pep_id`, each with exactly one accession string and one start list), so it moves to a per-peptide side table and the precursor table keeps no protein columns at all. Side tables are separate Arrow files in the `.poin` directory (an Arrow file has one schema, so tables with different columns and row counts cannot share a file). IDs are the value’s *rank in String sort order*, the convention `PrecursorTextIds` already uses, so sorting IDs sorts the text.

v1 column	v2	Side table	Est. at 300M
<code>koina_sequence</code>	dropped	—	0
<code>collision_energy</code>	dropped	— (NCE stays in <code>config.json</code>)	0
<code>isotopic_mods,</code> <code>isotope_mods</code>	written only with isotope groups	—	0
<code>sequence</code>	packed 5-bit residues, variable length	—	3.1 GB
<code>structural_mods</code>	one list of 2-byte entries (6-bit position, 2-bit site, 8-bit mod-name ID)	<code>precursor_mod_names.arrow:</code> ≤255 names	1.4 GB
<code>accession_numbers,</code> <code>proteome_identifiers,</code> <code>start_idx</code>	removed; looked up through <code>base_pep_id</code>	<code>precursor_peptides.arrow:</code> one row per <code>base_pep_id</code> (~62M) with accession-set ID, proteome ID, start positions; <code>precursor_accession_sets.arrow:</code> distinct accession sets, sorted, with member accession IDs; <code>precursor_accessions.arrow,</code> <code>precursor_proteomes.arrow:</code> distinct names	1.0 GB
<code>partner_precursor_idx</code>	UInt32 (row positions)	—	1.2 GB
numeric and one-byte columns	unchanged	—	9.6 GB
Total			≈16 GB

Sequences: packed residues

Every residue is one of 20 amino acids, so it fits in 5 bits (31 codes; room for U and O later). Codes are assigned in alphabetical order, starting at 1, and packed left-aligned with 0 as padding, so comparing two packed sequences byte by byte gives **String** order: sorting precursors by sequence, and the sequence ranks **PrecursorTextIds** needs, need no strings. Decoding to a **String** is a loop over at most 50 codes (the streaming builder already does this millions of times per second). Length, sulfur count and the other per-sequence features already have their own columns.

Bytes per precursor	Human 8–12mers	Yeast tryptic (7–34, mean 14)
String (today, incl. 4-byte offset)	14	18
ID into a table of distinct sequences	~8.9	~10
Packed, fixed width per library ($\lceil 5L_{\max}/8 \rceil$ bytes)	8	22
Packed, variable length (4-byte offset + $\lceil 5L/8 \rceil$)	~10	~13

Decision: variable length only. A fixed width is set by the library’s longest allowed peptide, so it saves 2 bytes for 8–12mers but is larger than today’s strings for tryptic libraries. Variable-length packing is smaller than strings for every library, with one format and one decoder. Stored as an Arrow binary column (**sequence_packed**) of $\lceil 5L/8 \rceil$ bytes; the length is the number of non-zero 5-bit codes (no residue has code 0), so no length byte is stored and byte order stays **String** order.

Modifications: one list of compact entries

Measured: 0.39 mods per precursor (0.16 variable Met-ox, 0.23 fixed carbamidomethyl), 68% of precursors have none, at most 8, and 2 distinct mod names (human 8–12mers; yeast tryptic is the same: 0.38 per precursor).

Mods encoding (human 8–12mers)	Bytes per precursor
String today, e.g. (5,M,Unimod:35)	9.8
Three list columns: positions, residues, mod-name IDs	~13
One list of 2-byte entries (position, site, mod-name ID)	~4.8
ID into a table of distinct mod combinations	2

Every Arrow list column stores a 4-byte offset per row, even for rows without mods, and two thirds of rows have none, so three separate list columns cost more than today’s strings. One list with all three fields per entry shares a single offset. Each entry packs into a **UInt16**:

- 6 bits: position (1–63; the streaming builder’s limit is 50 residues);
- 2 bits: site, *residue* (the residue is **sequence[position]**, so it is not stored), *N-term* or *C-term* (terminal mods are labelled **n** / **c** rather than with a residue, e.g. after decoy shuffling);
- 8 bits: mod-name ID into **mod_names** (Unimod name, mass), at most 255 names per library.

Entries are kept in today’s order (position, then name), so decoding reproduces today’s **structural_mods** string exactly, and readers that regex-parse that string per precursor (MainSearch features, iRT refinement, protein inference) can use the entries directly. A table of distinct combinations would be smaller still, but its size depends on how many mods a library allows, and every reader would need an extra lookup.

Protein information: side tables keyed by **base_pep_id**

- **precursor_peptides.arrow**, one row per **base_pep_id** (about 62M rows instead of 300M): **accession_set_id::UInt32**, **proteome_id::UInt16**, **start_idx::List{UInt32}**.
- **precursor_accession_sets.arrow**: the distinct accession sets in sort order (their row is their ID), each as text and as its sorted member accession IDs: what CV folds, protein groups and **PrecursorTextIds** need, without splitting a string per precursor.
- **precursor_accessions.arrow**, **precursor_proteomes.arrow**, **precursor_mod_names.arrow**: the distinct names (and each mod’s mass).

Why not Arrow compression or dictionary encoding

- Compressed columns cannot be memory-mapped; SearchDIA would decompress the whole table into RAM. Arrow dictionary encoding would give the same per-row IDs with the dictionary inside the file.
- But across record batches dictionary encoding relies on dictionary deltas, which are untested with Arrow.jl here, and its IDs would not be sorted ranks. Explicit ID columns and side tables keep everything memory-mapped and under our control.

4 Search-time memory

Measure before and after on the 10% (31M) and 100% (300M) mock libraries: `loadSpectralLibrary` time and peak RSS, and `getPrecursorTextIds` time and peak RSS. Expected: the per-precursor `String` vector and the `Dictionary` over accession keys in `StandardLibraryPrecursors` disappear (CV folds become a per-set array indexed by `accession_set_id`), and text IDs become column reads.

5 Compatibility

- The table carries a schema version in its Arrow metadata (`pioneer_precursor_schema = 2`); absent means v1.
- `LibraryPrecursors` gets a v2 implementation. The getters (`getSequence`, `getStructuralMods`, `getAccessionNumbers`, ...) return read-only vectors that decode a packed sequence, mod entries or an ID to today's string, so every existing reader keeps working; hot readers switch to the compact forms.
- v1 libraries load through today's code, unchanged. SearchDIA outputs (CSV and Arrow) keep the same text columns.
- CV-fold assignment must give the same folds: it seeds once and draws per distinct accession set in first-seen order, so the v2 path must draw in that same first-seen order, not ID order.

6 Phases

Each phase must give identical SearchDIA results (precursors and protein groups, long and wide) on the E. coli integration library and on the 3P library with 3 Olsen Astral files (RIS), before moving on.

1. **Drop dead columns:** `koina_sequence`, `collision_energy`, and the isotope-mod columns without isotope groups (readers treat an absent column as all missing). About -9 GB.
2. **v2 loading:** schema version, side-table loader, the v2 `LibraryPrecursors` and its getters, CV folds and `PrecursorTextIds` from stored IDs; tested on v2 tables converted from v1 libraries.
3. **v2 build:** the streaming builder writes packed sequences, mod entries, ID columns and side tables directly (it already holds sequences as packed codes and mods as (position, mod ID) pairs, and has the distinct peptides and protein sets in memory). About -9 GB more.
4. **Hot readers:** MainSearch features, iRT refinement and protein inference use packed sequences, mod entries and IDs instead of parsing strings. Timing measured per step.
5. **Scale:** rebuild the 300M mock library; record table size, build time and peak memory, and library load time and peak memory.

7 Decisions (9 October 2026)

1. `collision_energy` is dropped outright.
2. Sequences use variable-length packing only; no fixed-width form.
3. Side tables are separate Arrow files in the `.poin` directory.